

Debugging Lessons — Personal Notes

A running list of bugs I've actually made, grouped by the underlying concept, so I stop repeating them. Written from the NutriPlan food-log debugging session.

1. JavaScript syntax & typo traps

= vs : inside an object literal

```
// ✗ Wrong — this is destructuring syntax, illegal outside
let obj = {
  total += value,
};
// ✓ Right
let obj = {
  total: value,
};
```

Concept: `{ key: value }` and `{ key = defaultValue }` look similar but mean completely different things. The second form is only legal in *destructuring* patterns (`const { key = default } = source`). Using `=` in a plain object literal throws `SyntaxError: Invalid shorthand property initializer`.

+= vs +=

```
total += value;    // ✗ means: total = (+value) → OVERWR  
total += value;    // ✓ means: total = total + value
```

Concept: `+=` is just an assignment (`=`) followed by unary plus (`+value`, which just coerces to number). It silently does the *wrong thing* instead of throwing an error — the most dangerous kind of typo.

`!=` / `=` ! **VS** `!==`

```
if (x != null) { }    // ✗ assigns x = (!null) = true.  
if (x !== null) { }   // ✓ actual comparison
```

Concept: Same trap as above — looks like a comparison, is actually an assignment. Always double check: is there a `=` where I meant `==`/`===`?

Missing `{ }` after `if`

```
if (result.isConfirmed)  
  doThing();  
  doOtherThing();    // ✗ NOT inside the if! Runs uncon
```

Concept: Without braces, `if` only controls the **single next statement**. Everything after that runs no matter what. **Always use braces**, even for one-liners, to avoid this class of bug entirely.

Unescaped quotes inside inline HTML attributes

```
// ✗ the inner "meal" quotes prematurely close the attri  
onclick="deleteProduct(${i},"meal") "  
// ✓ use single quotes inside a double-quoted attribute  
onclick="deleteProduct(${i}, 'meal') "
```

Concept: Template literals don't know about HTML attribute boundaries — if the attribute is opened with `"`, any `"` inside it (even from a string

argument) breaks the HTML, not just the JS.

Typo'd localStorage key (trailing space, inconsistent spelling)

```
localStorage.setItem("mealsArray ", ...)    // ✗ trailing  
localStorage.getItem("mealsArray")          // won't find
```

Concept: String keys must match **exactly** — a stray space or a spelling inconsistency (protien vs protein) creates a silent new key/property instead of erroring. Always keep property/key names consistent across every file that reads/writes them.

2. State, timing & "stale data" bugs

Cloning a value at import time instead of reading it live

```
// ✗ captured once, at module load — never updates again  
export let logMealObj = structuredClone(logMealObject);  
  
// ✓ read the live imported binding at the moment you access it  
if (logMealObject !== "") {  
  mealsArray.push(structuredClone(logMealObject));  
}
```

Concept: ES module `let/var` exports are live-bound *if you reference the imported name directly*, but the moment you copy that value into a **new local variable**, you've frozen a snapshot. Only clone data at the point of use, not at setup time.

Calling the "calculate" step before the "load" step

```
// ✗ wrong order – calculates on empty arrays, then load
calculateAllNutrition();
showAllNutrition();
confirmData(); // loads real data – but nothing recalculates

// ✓ load first, then calculate, then render – all inside
function confirmData(){
  mealsArray = JSON.parse(localStorage.getItem("mealsArray"));
  productsArray = JSON.parse(localStorage.getItem("productsArray"));
  calculateAllNutrition();
  showAllNutrition();
}
```

Concept: Data pipelines have a required order: **load** → **calculate** → **render**. Calling these out of order, or only once at page-load instead of after every state change, leaves the UI showing stale results.

A function that reloads old data over its own fresh changes

```
function deleteProduct(index){
  productsArray.splice(index, 1); // ① fresh change,
  confirmData(); // ② re-reads OLD
  localStorage.setItem("productsArray", JSON.stringify(productsArray));
}
```

Concept: Never call a "load from storage" function *after* making an in-memory change you haven't saved yet — it'll overwrite your change with the old saved version. Correct order: **mutate** → **save** → **recalculate** → **render**. Nothing else.

Requiring two independent things to *both* exist before loading either

```
// ✗ if productsArray was never saved, mealsArray never loads
if (localStorage.getItem("mealsArray") != null && localStorage.getItem("productsArray") != null) {
  // ...
}
```

```

    mealsArray = JSON.parse(localStorage.getItem("mealsArra
    productsArray = JSON.parse(localStorage.getItem("produc
  }

  // ✓ check and load each independently
  if (localStorage.getItem("mealsArray") !== null) {
    mealsArray = JSON.parse(localStorage.getItem("mealsArra
  }
  if (localStorage.getItem("productsArray") !== null) {
    productsArray = JSON.parse(localStorage.getItem("produc
  }

```

Concept: A `&&` B as a gate means *neither* loads unless *both* are true. If two data sources are actually independent of each other, don't couple their loading with `&&`.

Empty-string / sentinel values sneaking past a null check

```

// ✗ "" passes both checks — it's neither null nor undef
if (value !== null && value !== undefined) { }

// ✓ explicitly exclude the sentinel value too
if (value !== null && value !== undefined && value !== "")

```

Concept: "Empty" and "absent" are different states. If a variable's default/cleared value is "", a null/undefined check alone won't catch it — decide what your **actual sentinel value** is and check for that specifically.

3. Scope bugs

const/let inside a block shadowing an outer variable

```
function build(){
  let result = "";
  if (empty) {
    result = "empty state";      // ✓ assigns the OUTER
  } else {
    const result = "has data";  // ✗ creates a NEW vari
  }                             //      thrown away here
  return result;               // always "" in the else-branch case
}
```

Concept: `let/const` are block-scoped. Re-declaring the same name inside a nested `{ }` doesn't update the outer variable — it silently creates a second, unrelated one that dies at the closing brace. If you mean to update an existing variable, don't redeclare it — just assign (`result = ...`, no keyword).

4. DOM & rendering bugs

Height/percentage collapsing because the wrong element got the constraint

```
<!-- Parent has a fixed height; child uses h-full (100% o
<div class="h-1.5 bg-gray-200">           <!-- track -->
  <div class="h-full bg-emerald-500" style="width: 66%"><
</div>
```

Concept: `height: 100%` (or Tailwind's `h-full`) only works if the **parent** has an explicit height. If the bar isn't showing, inspect the *parent's* computed height first, not just the fill element's classes.

`overflow-hidden` on the wrong element

```
<!-- ✗ overflow-hidden on the CHILD does nothing — it ha
```

```
<div class="w-full bg-gray-200">
  <div class="bg-purple-500 overflow-hidden" style="width: 156%">
    <!-- ✓ overflow-hidden belongs on the PARENT, to clip the children
  <div class="w-full bg-gray-200 overflow-hidden">
    <div class="bg-purple-500" style="width: 156%"></div>
  </div>
```

Concept: `overflow` controls how an element treats *its own children* that spill outside its box — it does nothing to control how the element itself relates to *its parent*. Also: cap percentage-based widths at `Math.min(value, 100)` so the underlying number is correct, not just visually clipped.

Forgetting to loop over a second data source

```
// ✗ only iterates mealsArray — productsArray never gets iterated
for (let i = 0; i < mealsArray.length; i++) { ... }

// ✓ loop both, append to the same output string
for (let i = 0; i < mealsArray.length; i++) { ... }
for (let i = 0; i < productsArray.length; i++) { ... }
```

Concept: When a UI section is meant to show two related-but-separate data sources, double check every rendering loop actually covers both — it's easy to build and test against just the first one.

Hardcoded placeholder values left in "finished" template code

```
// ✗ static numbers baked into every rendered card, regardless of data
<p>2152</p> <span>111g P</span>

// ✓ pull from the real object
<p>${item.calories}</p> <span>${item.protein}g P</span>
```

Concept: Placeholder/mock values used while building the UI are easy to forget to wire up to real data once the rest of the logic is working. Grep for suspicious literal numbers before calling a template "done."

5. Event handling bugs

Attaching a listener to an element that doesn't exist yet

```
// ✗ if this runs before showProducts() has rendered any  
const btn = document.getElementById("clear-foodlog");  
btn.addEventListener("click", ...); // TypeError: Cannot
```

Concept: Any code that queries the DOM must run *after* that DOM exists. If a button is created dynamically via `innerHTML`, code that tries to attach a listener to it before that render step will simply fail to find it.

`innerHTML` = destroying previously-attached listeners

```
container.innerHTML = newHtml; // wipes out ALL child e
```

Concept: Setting `innerHTML` doesn't just change what's *displayed* — it destroys the old DOM nodes entirely and creates brand new ones from scratch. Any listener attached directly to an element inside that container is gone the next time that container re-renders, even if the new HTML looks identical.

The fix: event delegation on a stable parent

```
// ✓ attach ONCE, to a container that is never itself re  
const container = document.getElementById("allProducts-gr  
container.addEventListener("click", function(e) {  
  const btn = e.target.closest("#clear-foodlog");
```

```
    if (!btn) return;           // click wasn't on the button
    // ... handle it
  });
```

Concept: Instead of attaching a listener to something that gets rebuilt often, attach it once to a stable ancestor and use `e.target.closest(selector)` to check what was actually clicked. This listener survives every re-render because the *container* itself is never replaced — only its contents are.

Inline `onclick="fn () "` calling a function that's scoped to a module

```
// foodLog.js — an ES module
function deleteProduct(i, type){ ... }
// ✗ inline onclick runs in GLOBAL scope; module function is not visible
```

```
// ✓ explicitly expose it, once, at module top-level
window.deleteProduct = deleteProduct;
```

Concept: `<script type="module">` code is scoped to the module — nothing is added to `window` automatically. Inline event handler attributes (`onclick`, `onchange`, etc.) always execute in the global scope, so they can only see functions that were explicitly attached to `window`. (Better long-term fix: avoid inline handlers altogether and use `addEventListener` + event delegation instead.)

Quick pre-flight checklist for next time

- ☐ Any `if` without `{ }`? Add braces even for one-liners.
- ☐ Any `=` where I meant `==/===`, or `+=` where I meant `+=`?
- ☐ Does every property name match exactly across every file that uses it?
- ☐ Am I loading data *before* calculating/rendering from it?

- ☐ After mutating an array, am I saving *before* any function that reloads from storage?
- ☐ Did I redeclare a variable with `let/const` inside a block instead of reassigning it?
- ☐ Does a percentage/width get capped at 100 and does `overflow-hidden` sit on the *parent*?
- ☐ Are all relevant arrays looped over in render functions, not just the first one?
- ☐ Any leftover hardcoded/placeholder values that should be dynamic?
- ☐ Are event listeners attached after the target exists, and do they survive re-renders (delegation on a stable parent)?
- ☐ Do inline `onclick` handlers reference functions actually exposed on `window`?